Chetna Kurzekar CS23174

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
path="/content/drive/MyDrive/ML datasets/pulsar_stars (1).csv"
df=pd.read_csv(path)
df.head()
```

	Mean of the integrated profile	Standard deviation of the integrated profile	Excess kurtosis of the integrated profile	Skewness of the integrated profile	Mean of the DM-SNR curve	Standard deviation of the DM-SNR curve	Excess kurtosis of the DM-SNR curve	Skewness of the DM-SNR curve	target_class
0	140.562500	55.683782	-0.234571	-0.699648	3.199833	19.110426	7.975532	74.242225	0
1	102.507812	58.882430	0.465318	-0.515088	1.677258	14.860146	10.576487	127.393580	0
2	103.015625	39.341649	0.323328	1.051164	3.121237	21.744669	7.735822	63.171909	0
3	136.750000	57.178449	-0.068415	-0.636238	3.642977	20.959280	6.896499	53.593661	0

Next steps: Generate code with df New interactive sheet

```
df.columns = df.columns.str.strip()
```

```
df.rename(columns={'Mean of the integrated profile': 'Mean integrated profile', 'Standard deviation of the integrated profi
```

```
print(df.columns.tolist())
```

```
['Mean integrated profile', 'standard integrated profile', 'Excess kurtosis of the integrated profile', 'Skewness of the int
```

```
print(df.columns.str.contains(' ').any())
```

```
True
```

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17898 entries, 0 to 17897
Data columns (total 9 columns):
 #   Column                                     Non-Null Count  Dtype
---  ------                                     --------------  -----
 0   Mean integrated profile                    17898 non-null  float64
 1   standard integrated profile                17898 non-null  float64
 2   Excess kurtosis of the integrated profile  17898 non-null  float64
 3   Skewness of the integrated profile         17898 non-null  float64
 4   Mean of the DM-SNR curve                   17898 non-null  float64
 5   Standard deviation of the DM-SNR curve     17898 non-null  float64
 6   Excess kurtosis of the DM-SNR curve        17898 non-null  float64
 7   Skewness of the DM-SNR curve               17898 non-null  float64
 8   target_class                               17898 non-null  int64
dtypes: float64(8), int64(1)
memory usage: 1.2 MB
None
```

```
df.shape
```

```
(17898, 9)
```

```
df.columns
```

```
Index(['Mean integrated profile', 'standard integrated profile',
       'Excess kurtosis of the integrated profile',
       'Skewness of the integrated profile', 'Mean of the DM-SNR curve',
       'Standard deviation of the DM-SNR curve',
```

```
       'Excess kurtosis of the DM-SNR curve', 'Skewness of the DM-SNR curve',
       'target_class'],
      dtype='object')
```

```
df.isnull().any().any()
```

```
np.False_
```

```
from sklearn.model_selection import train_test_split
X = df.drop(columns=['target_class'])
y = df['target_class']
X_train,X_test,y_train,y_test = train_test_split(X,y,train_size=0.7,test_size=0.3,random_state=42)
X_train.shape,X_test.shape
```

```
((12528, 8), (5370, 8))
```

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
X_train
```

```
array([[ 0.39398903,  0.27225212, -0.09759786, ..., -0.40338108,
        -0.19481754, -0.37780071],
       [-1.32725743, -1.52127273,  0.21878252, ..., -0.44578706,
         0.23837256, -0.02857034],
       [ 0.76130595,  0.98636783, -0.3245843 , ..., -0.2004973 ,
         0.038556  , -0.29531727],
       ...,
       [ 0.19435365, -0.48635278,  0.17242235, ..., -0.75001383,
         1.92294752,  1.91898259],
       [ 0.95911539,  0.50175114, -0.53401036, ..., -0.24457614,
        -0.20616758, -0.4040674 ],
       [ 0.37725131,  0.58093396, -0.12507761, ..., -0.74159513,
         2.05677923,  2.08022728]])
```

```
y_train
```

	target_class
12223	0
1919	0
2753	0
17220	0
12110	0
...	...
11284	0
11964	0
5390	0
860	0
15795	0

12528 rows × 1 columns

dtype: int64

```
from sklearn.metrics import confusion_matrix,accuracy_score, classification_report
```

```
from sklearn.svm import SVC
svc_model = SVC(random_state=42)
svc_model.fit(X_train, y_train)
y_pred = svc_model.predict(X_test)
print('Confusion Matrix')
cm=confusion_matrix(y_pred,y_test)
print(confusion_matrix(y_pred,y_test))
```

```
Confusion Matrix
[[4863   92]
 [  21  394]]
```

```
print('Accuracy Score')
print(accuracy_score(y_pred,y_test))
```

```
Accuracy Score
0.9789571694599628
```

```
print('Confusion Matrix')
cm=confusion_matrix(y_pred,y_test)
print(confusion_matrix(y_pred,y_test))
```

```
Confusion Matrix
[[4863   92]
 [  21  394]]
```

```
import numpy as np
cm = np.array([[4863, 92],
               [21, 394]])
accuracy = np.trace(cm) / np.sum(cm)
print(f"Accuracy: {accuracy:.4f}")
```

```
Accuracy: 0.9790
```

```
class_report = classification_report(y_test, y_pred, target_names=['0', '1'])
print("Classification Report:")
print(class_report)
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99      4884
           1       0.95      0.81      0.87       486

    accuracy                           0.98      5370
   macro avg       0.97      0.90      0.93      5370
weighted avg       0.98      0.98      0.98      5370
```

```
linear_svc=SVC(kernel='linear')
linear_svc.fit(X_train,y_train)
y_pred=linear_svc.predict(X_test)
print('model accuracy score:')
print(accuracy_score(y_pred,y_test))
```

```
model accuracy score:
0.9787709497206704
```

```
poly_svc=SVC(kernel='poly')
poly_svc.fit(X_train,y_train)
y_pred=poly_svc.predict(X_test)
```

```
sigmoid_svc=SVC(kernel='sigmoid')
sigmoid_svc.fit(X_train,y_train)
y_pred=sigmoid_svc.predict(X_test)
print(confusion_matrix(y_test,y_pred))
print(classification_report(y_test,y_pred))
print(accuracy_score(y_test,y_pred))
```

```
[[4556  328]
 [ 331  155]]
              precision    recall  f1-score   support

           0       0.93      0.93      0.93      4884
           1       0.32      0.32      0.32       486

    accuracy                           0.88      5370
   macro avg       0.63      0.63      0.63      5370
weighted avg       0.88      0.88      0.88      5370

0.8772811918063315
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
cm=confusion_matrix(y_test,y_pred)
print(confusion_matrix(y_test,y_pred))
print(accuracy_score(y_test,y_pred))
print(classification_report(y_test,y_pred))
```

```
[[4556  328]
 [ 331  155]]
0.8772811918063315
              precision    recall  f1-score   support

           0       0.93      0.93      0.93      4884
           1       0.32      0.32      0.32       486
```

```
    accuracy                           0.88      5370
   macro avg       0.63      0.63      0.63      5370
weighted avg       0.88      0.88      0.88      5370
```